



Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Programación 3

Trabajo Práctico Integrador

Profesor: Cinthia Rigoni

Tutora: Renzo Sosa

Alumno: Lucas Daniel Fredes

Comision: 3

Link del video:

<https://drive.google.com/drive/folders/1jcK9JwNpYKI-Jy9-R0CT0KI-9jfNrIWb?usp=sharing>



1. Índice

1. Marco Teórico	Pág. 3
○ Ecosistema Backend: Java, JPA e Hibernate	
○ Base de Datos Relacional H2	
○ Ecosistema Frontend: Vite y TypeScript	
2. Decisiones Técnicas y Arquitectura	Pág. 4
○ Arquitectura del Backend y Patrón Repository	
○ Gestión de Ciclo de Vida y Transacciones ACID	
○ Mecanismo de Soft Delete (Baja Lógica)	
○ Arquitectura Frontend y Capa de Fetch Desacoplada	
3. Dificultades y Soluciones	Pág. 8
○ Obstáculo 1: Sincronización del estado global del carrito ante la recarga de la página.	
○ Obstáculo 2: Control estricto de accesos y renderizado de componentes según el Rol (CLIENT / ADMIN).	
4. Bibliografía y Webgrafía	Pág. 9



2. Marco Teórico

Ecosistema Backend: Java, JPA e Hibernate

El núcleo del servidor se construyó sobre Java, utilizando la especificación JPA (Jakarta Persistence API). JPA actúa como una capa de abstracción orientada a objetos que define cómo mapear objetos Java a tablas de bases de datos relacionales. Como proveedor de persistencia (ORM) se utilizó Hibernate, el cual se encarga de traducir las operaciones orientadas a objetos y las consultas estructuradas en lenguaje JPQL hacia las sentencias SQL nativas del motor subyacente.

Base de Datos Relacional H2

Para la capa de almacenamiento de datos se optó por el motor H2 Database Engine en modo embebido de archivo físico. H2 es un motor relacional en Java de alto rendimiento que permite configurarse mediante rutas relativas (`jdbc:h2:file:./data/jpa_db`). Esto garantiza que los datos persistan de forma local en el proyecto sin depender de un servidor externo de base de datos, facilitando la portabilidad del sistema.

Ecosistema Frontend: Vite y TypeScript

La interfaz gráfica de usuario web se diseñó utilizando TypeScript, el cual añade tipado estático fuerte sobre JavaScript, minimizando errores lógicos y de asignación en tiempo de desarrollo. Como empaquetador de la aplicación se adoptó Vite, un *build tool* de última generación que optimiza la velocidad de recarga en caliente (HMR - Hot Module Replacement) y el empaquetado final de los recursos del navegador.



3. Decisiones Técnicas y Arquitectura

Arquitectura del Backend y Patrón Repository

Para la persistencia, se implementó el patrón Repository mediante una clase genérica abstracta `BaseRepository<T>`. Esta clase encapsula las operaciones CRUD fundamentales (guardar, buscar por ID, listar activos) interactuando directamente con el `EntityManagerFactory`.

Luego, cada entidad cuenta con su repositorio específico (`UsuarioRepository`, `CategoriaRepository`, etc.), extendiendo de la base. Esto permite aislar por completo el código JPQL de la lógica de control. Las consultas avanzadas personalizadas se escribieron usando JPQL incrustado, incluyendo comentarios de bloque nativos (`/* ... */`) para documentar el comportamiento de los JOINS y filtros aplicados.

Gestión de Ciclo de Vida y Transacciones ACID

En procesos complejos y críticos para el negocio, como el Alta de Pedido Atómico (`ejecutarAltaPedidoAtómico`), se decidió prescindir de la delegación automática de Hibernate y tomar un control manual estricto del ciclo de vida transaccional mediante `EntityTransaction`.

El flujo garantiza la atomicidad y consistencia (ACID) del sistema abriendo un bloque try-catch-finally: se sincronizan los productos mediante `em.find()`, se valida el stock concurrente en caliente, y solo si todas las líneas del carrito son válidas, se descuenta el stock y se confirma con `tx.commit()`. Ante cualquier excepción o falta de inventario, el bloque catch ejecuta un `tx.rollback()`, garantizando que la base de datos nunca quede en un estado inconsistente o con lecturas sucias.

Mecanismo de Soft Delete (Baja Lógica)

Con el fin de preservar la integridad referencial histórica y cumplir con la rúbrica de evaluación, se implementó el patrón de Soft Delete a nivel entidad.

Arquitectura Frontend y Capa de Fetch Desacoplada



En la Parte 1 (Frontend), se aplicó un principio estricto de desacoplamiento de datos. Toda la información de categorías, productos y usuarios se almacena en archivos estáticos `.json` simulados.

El acceso a estos datos se centralizó en módulos de servicio dedicados (services) utilizando la API nativa `fetch()`. Esta decisión de arquitectura permitió construir una interfaz reactiva autónoma (con login de roles, carritos dinámicos y paneles de administración) independiente del estado del servidor Java. Al estar la capa de servicios completamente aislada, conectar la web a una API REST real en un sprint posterior se reduce a modificar las URLs de los `fetch()`, sin necesidad de alterar el maquetado ni la lógica de componentes.

FoodStore

Carrito 0 Catalogo Mis Pedidos

CATEGORIAS

Todos los productos

Pizzas

Hamburguesas

Bebidas

Empanadas

Papas Fritas

Ordenar por... ▾

Se encontraron 23 productos.

Productos



Hamburguesas
Hamburguesa Clásica
Carne 150g, lechuga, tomate, cebolla morada y mayonesa
\$4.500 STOCK: 48 u.

Ver Detalles Agregar



Hamburguesas
Hamburguesa Doble
Doble carne 150g c/u, cheddar, panceta y salsa barbacoa
\$6.200 STOCK: 38 u.

Ver Detalles Agregar



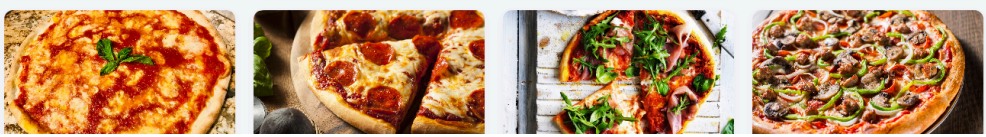
Hamburguesas
Hamburguesa Completa
Carne 150g, huevo, jamón, queso, lechuga y tomate
\$5.500 STOCK: 30 u.

Ver Detalles Agregar



Hamburguesas
Hamburguesa Veggie
Medallón de lentejas y quinoa, rúcula, tomate confitado y hummus
\$5.000 STOCK: 20 u.

Ver Detalles Agregar



Tu Pedido



Hamburguesa Clásica
Subtotal: \$4.500

- 1 +

Eliminar



Hamburguesa Doble
Subtotal: \$6.200

- 1 +

Eliminar



Pizza Mozzarella
Subtotal: \$3.800

- 1 +

Eliminar

Resumen

Subtotal\$14.500

Envío\$500

Total\$15.000

Datos de Entrega

Dirección de Entrega *
Ej: Av. Siempreviva 742

Teléfono de Contacto *
Ej: 1123456789

Método de Pago *
Seleccione una opción

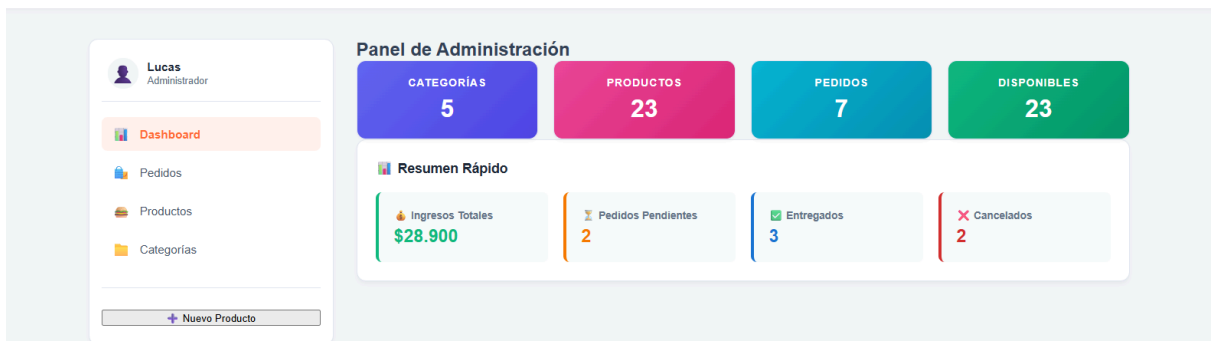
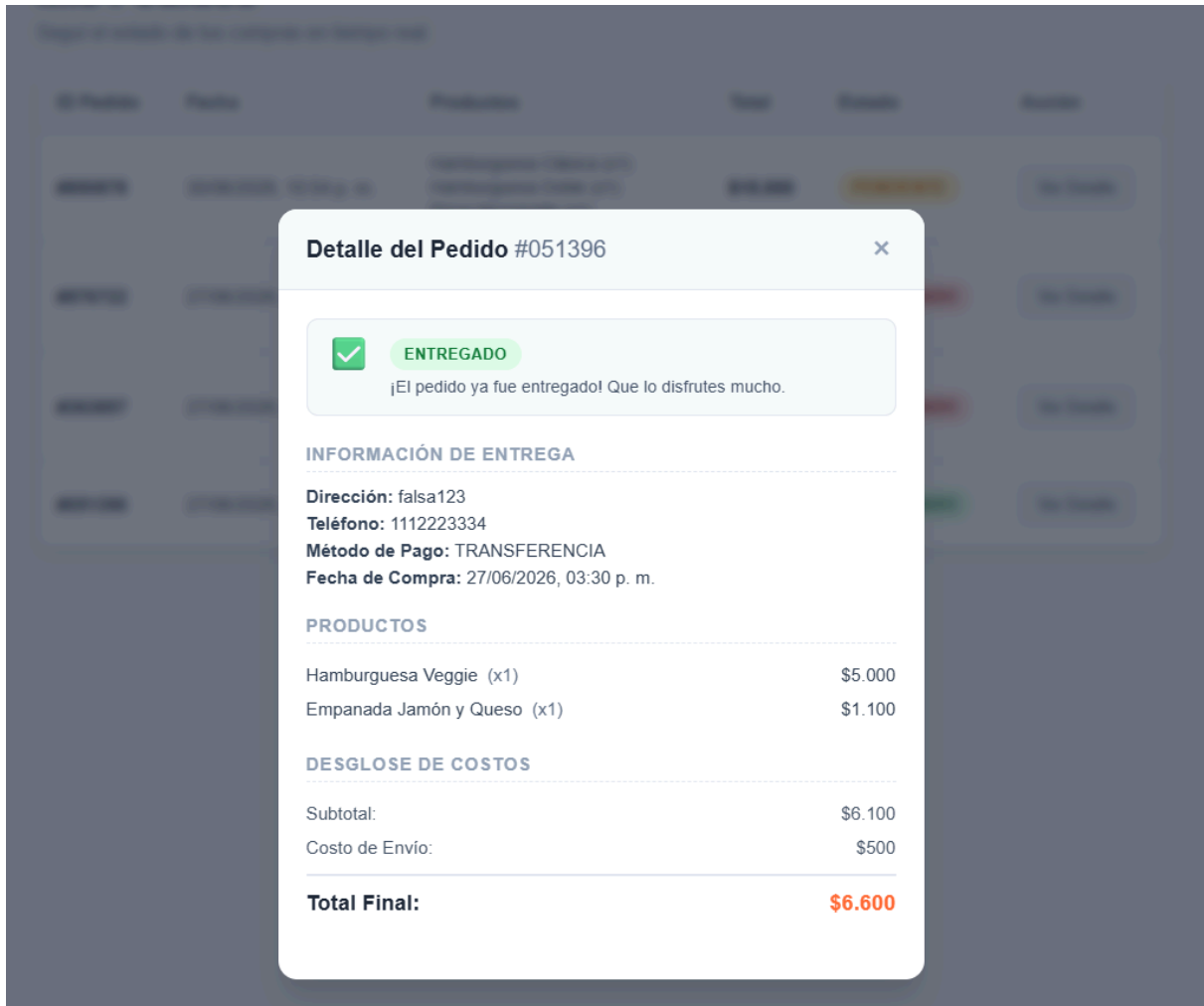
Finalizar compra

Vaciar carrito

Mis Pedidos

Seguí el estado de tus compras en tiempo real.

ID Pedido	Fecha	Productos	Total	Estado	Acción
#850878	30/06/2026, 10:54 p. m.	Hamburguesa Clásica (x1) Hamburguesa Doble (x1) Pizza Mozzarella (x1)	\$15.000	PENDIENTE	Ver Detalle
#576722	27/06/2026, 03:39 p. m.	Hamburguesa Veggie (x2) Empanada Jamón y Queso (x1) Sprite 500ml (x2)	\$14.600	CANCELADO	Ver Detalle
#363857	27/06/2026, 03:36 p. m.	Empanada de Verdura (x1) Empanada Jamón y Queso (x1) Agua Mineral 500ml (x1)	\$3.700	CANCELADO	Ver Detalle
#051396	27/06/2026, 03:30 p. m.	Hamburguesa Veggie (x1) Empanada Jamón y Queso (x1)	\$6.600	ENTREGADO	Ver Detalle





4. Dificultades y Soluciones

Obstáculo 1: Sincronización del estado global del carrito ante la recarga de la página.

Al realizar pruebas en el flujo de cliente, se detectó que si el usuario refrescaba el navegador (F5) o navegaba manualmente hacia el detalle de un producto y volvía, los ítems añadidos al carrito de compras se borraban por completo y el contador superior volvía a cero. Esto ocurría porque el estado del carrito se gestionaba únicamente de forma volátil en la memoria de la aplicación web.

Se diseñó un mecanismo de persistencia local mediante la API `localStorage` del navegador. Se programó para que, cada vez que se agregue, modifique o elimine un producto del carrito, el estado actual se serialice en formato JSON y se guarde en el almacenamiento local. Al inicializar la aplicación, el servicio del carrito realiza una lectura preventiva de `localStorage`; si encuentra datos previos, los restaura de inmediato de forma reactiva, logrando que la experiencia de usuario sea consistente y no pierda su orden de compra.

Obstáculo 2: Control estricto de accesos y renderizado de componentes según el Rol (CLIENT / ADMIN).

Inicialmente, la estructura no detectaba bien el rol de cliente o admin y se podía navegar a la pantalla de administrador como cliente y comprar productos de catálogo desde el rol de admin.

Se implementó una lógica de enrutamiento y renderizado condicional basado en roles. Al procesar el inicio de sesión desde el servicio de autenticación, el objeto del usuario activo se almacena con su propiedad `rol` correspondiente. En el componente de navegación principal, se involucraron las secciones críticas (Dashboard, CRUD de Categorías, etc.) con validaciones explícitas que comprueban si el rol es estrictamente ADMIN. Si un CLIENT intenta forzar el ingreso, la interfaz lo redirige automáticamente al catálogo o renderiza un componente de "Acceso Denegado", garantizando el aislamiento de funciones.



5. Bibliografía / Webgrafía

Eclipse Foundation. (s.f.). *Jakarta Persistence (JPA) - Official Documentation*. Jakarta EE. <https://jakarta.ee/specifications/persistence/>

Hibernate. (s.f.). *Hibernate ORM User Guide*. <https://hibernate.org/orm/documentation/>

H2 Database. (s.f.). *H2 Database Engine - Tutorial & Connection Modes*. <https://www.h2database.com/html/tutorial.html>

Microsoft Corporation. (s.f.). *TypeScript Language Specification and Handbook*. TypeScript. <https://www.typescriptlang.org/docs/>

Vite. (s.f.). *Vite Guide - Backend Integration and Static Server Assets*. <https://vite.dev/guide/>